ACID-Compliant B+ Tree Database Engine

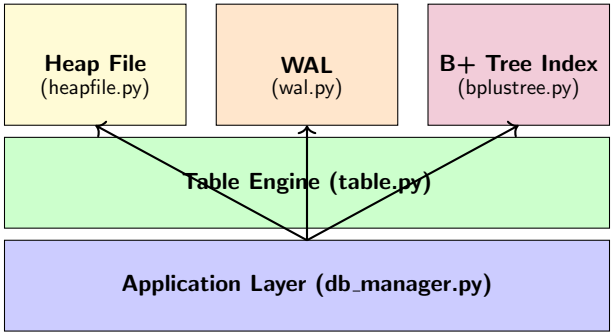
Module A: Transaction Management & Crash Recovery

CS 432 Databases - IIT Gandhinagar

April 14, 2026

Overview

System Architecture Overview



Layered Storage Design

Why Separate Layers?

- **Heap File**: Durable row store with deterministic slot-based addressing
- **WAL**: Write-ahead log for crash safety and atomicity
- **B+ Tree**: Fast in-memory index (pk $\rightarrow$ slot_id)
- **Metadata JSON**: Schema, allocator state, free list

Key Design Principle

Heap file is the **source of truth**. Indexes are rebuilt from heap on startup.

Heap File: Fixed-Width Binary Storage

Record Layout

```
[tombstone: 1 byte]  # 0x01=alive, 0x00=deleted
for each column:
    [null_flag: 1 byte]  # 0x01=NULL, 0x00=present
    [data: N bytes]      # Type-specific encoding
```

- **Fixed-width**: Enables O(1) slot addressing
- **Tombstone**: Logical deletion without physical compaction
- **Null flags**: Explicit NULL semantics
- **Type-aware encoding**: INT(4), FLOAT(8), BOOLEAN(1), CHAR/VARCHAR(N)

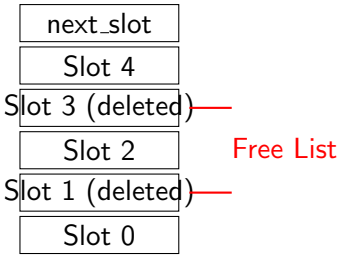
Slot Allocation Strategy

Two-Step Policy

1. Check free list (recycled slots)
2. If empty, allocate next_slot

Benefits

- Prevents unbounded file growth
- Improves locality
- Stable slot IDs for WAL/indexes



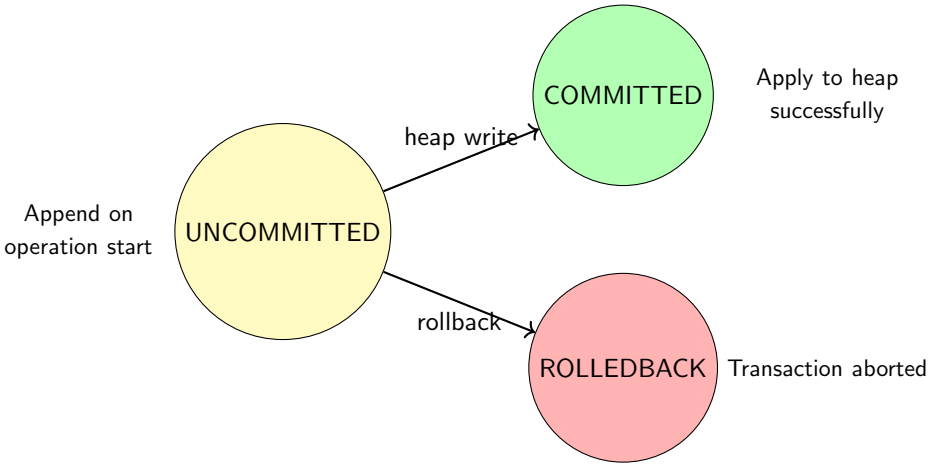
WAL Entry Structure

Binary Entry Format

```
[op: 1 byte]        # INSERT/UPDATE/DELETE/COMMIT_INTENT
[slot: 4 bytes]     # Heap slot ID
[txn_id: 4 bytes]   # Transaction ID (0=auto-commit)
[status: 1 byte]    # UNCOMMITTED/COMMITTED/ROLLEDBACK
[data: N bytes]     # Packed row bytes
```

- **Fixed entry size**: HEADER(9) + status(1) + record_size
- **O(1) status updates**: Direct seek to status byte
- **COMMIT_INTENT**: Special marker indicating user's intent to commit
- **Idempotent replay**: Same entry can be applied multiple times

WAL State Machine



Two Write Modes

Auto-Commit (Thread-Safe)

1. Append WAL (uncommitted)
2. Write to heap
3. Mark WAL committed
4. Update in-memory indexes
5. Save metadata

Lock: Per-table lock

Transaction Mode

1. Append WAL only (staged)
2. Update indexes immediately (dirty reads)
3. *Heap untouched*
4. On commit: apply all to heap
5. On rollback: mark rolledback, rebuild indexes

Lock: Single-threaded

Key Insight

Transaction staging updates indexes immediately for intra-transaction visibility, but keeps heap clean until commit.

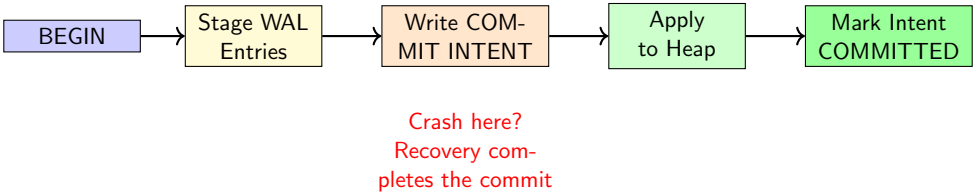
Transaction Lifecycle

```
txn = Transaction(users_table, orders_table)
txn.begin()  # Allocate unique txn_id

try:
    txn.insert(users_table, {"id": 1, "name": "Alice"})
    txn.insert(orders_table, {"id": 10, "user_id": 1})
    txn.commit()  # Apply all staged WAL entries to heap
except Exception:
    txn.rollback()  # Mark entries rolledback, rebuild index
    raise
```

- **Atomicity**: All-or-nothing across multiple tables
- **Isolation**: Single-threaded execution per transaction
- **Durability**: WAL persisted before heap writes

Commit Protocol with Intent Marker



Commit Intent Marker

Critical for recovery: marks user's intent to commit. If crash occurs after this marker but before completion, recovery will finish the commit. Without this marker, uncommitted transaction entries are skipped during recovery.

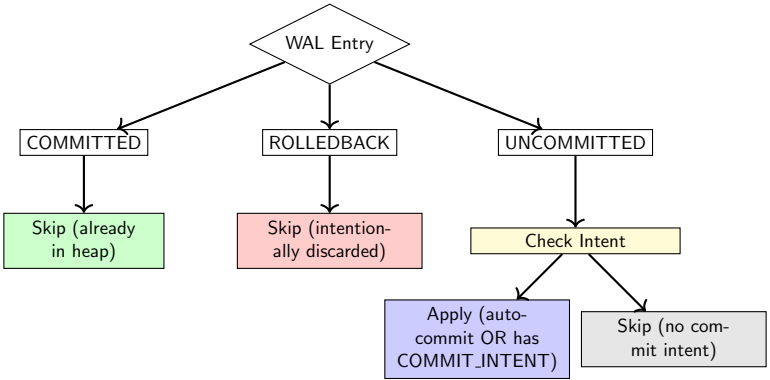
Startup Recovery Pipeline

1. **Scan WAL**: Collect re-applicable entries
   - UNCOMMITTED auto-commit entries (TXN_AUTO)
   - UNCOMMITTED entries belonging to transactions with COMMIT_INTENT marker
2. **Apply to Heap**: Idempotent replay of intended commits
3. **Skip**: COMMITTED (already applied) and ROLLEDBACK (intentionally discarded)
4. **Truncate Corrupted Tail**: Handle partial writes
5. **Rebuild Indexes**: From heap (source of truth)
6. **Delete WAL**: Start fresh session

Deterministic Convergence

System only replays operations the user intended to commit, ensuring ACID compliance.

Recovery Decision Tree



Foreign Key Constraints

Supported Actions

- **RESTRICT**: Prevent parent deletion if children exist
- **CASCADE**: Propagate delete/update to children
- **SET NULL**: Nullify child references
- **SET DEFAULT**: Set child references to default value

Implementation

- Parent existence checks use indexed paths
- Child lookups via secondary indexes (efficient fan-out)
- Deep cascade chains supported
- Transaction-aware: cascades rollback with parent operation

Secondary Indexes

Composite Key Design

Secondary index stores: `(column_value, pk)` in B+ tree

- **Allows duplicates**: Multiple rows with same value
- **Deterministic retrieval**: PK ensures unique ordering
- **Efficient range scans**: Equality and range queries
- **FK acceleration**: Fast child lookups for cascades

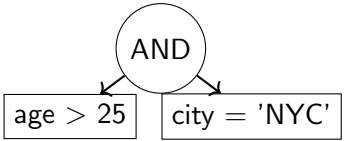
Example

Index on `user_id`: `(42, order_1)`, `(42, order_5)`, `(43, order_2)`

Condition Trees

Expression Tree

- **Leaf**: column op value
- **AND/OR/NOT**: Combinators
- **Operators**: $=$, $\neq$, $<$, $>$, $\leq$, $\geq$, IN, BETWEEN, IS NULL



Conservative Optimization

PK hints extracted only from safe AND branches. Final filtering always applied for correctness.

Join Strategies

Automatic Strategy Selection

1. **Index Nested-Loop**: Right join key is primary key
2. **Secondary-Index Join**: Right join key has secondary index
3. **Full Nested-Loop**: Fallback for unindexed predicates

- **LEFT JOIN**: Inject null-valued right columns when no match
- **Column hygiene**: Prefix right-table names on collision
- **WHERE filtering**: Applied after join merge

Atomicity Evidence

Transaction Rollback Scenarios

- Inserted records absent post-rollback
- Failure-injected branches show no partial visibility
- Cross-table inserts appear together or not at all

Test Case 2: Mid-Transaction Failure

- Insert into users, products, orders
- Force exception before commit
- **Result**: All tables unchanged, no partial state

Consistency Evidence

Schema & Referential Integrity

- Invalid PK updates rejected
- FK checks prevent dangling references
- Cascade/SET NULL maintain referential closure
- Deep-chain cascades leave valid terminal state

Test Case 3: 3-Table Order Transaction

- Atomic insert across users/products/orders
- FK violation attempts rejected
- Successful commit shows all constraints satisfied

Isolation Evidence

Concurrency Control

- Auto-commit: Per-table lock serialization
- Stress workload: 50 high-contention updates
- **Result**: Aggregate flag sum = 50/50 (no lost updates)

Test Case 4: Concurrent Transfers

- Original balance: 2,000,000
- 100+ concurrent transfer operations
- Final balance: 2,000,000
- **Invariant preserved**: No value creation or loss

Durability Evidence

Persistence Across Restarts

- Committed rows available after close/reopen
- Deleted rows remain absent across restart
- Transaction-committed orders persist after reload
- WAL replay reconstructs consistent state

Test Case 5: Restart Recovery

- Write committed, failed, and orphan staged work
- Simulate crash (close without cleanup)
- Restart and reload
- **Result**: Committed row persists, failed/orphan absent

File Organization

Core System

- `table.py`: Table engine
- `heapfile.py`: Binary storage
- `wal.py`: Write-ahead log
- `transactions.py`: Txn coordinator
- `bplustree.py`: In-memory index

Constraints & Query

- `column.py`: Schema validation
- `foreignkey.py`: Referential integrity
- `secondaryindex.py`: Non-PK indexes
- `conditions.py`: Expression trees
- `query.py`: Filter execution
- `join.py`: Join strategies

Design Rationale

Why Heap + WAL + In-Memory Index?

- **Heap**: Simple, deterministic slot-addressed persistence
- **WAL**: Operation-level idempotent replay
- **Index**: Reconstructed from committed row state
- **Trade-off**: Startup rebuild time vs. correctness simplicity

Alternative Rejected

Direct B+ tree node serialization increases crash complexity (in-flight splits/merges).

Failure Modes Addressed

1. Crash after WAL append, before heap write
2. Crash after heap write, before WAL status flip
3. Crash during multi-entry transaction commit
4. Truncated WAL tail (interrupted writes)
5. Process restart with stale in-memory index

Recovery Guarantee

For each failure class, startup replay + index rebuild converges to valid state.

Operation Complexity

Operation	Time Complexity	Notes
Insert	$O(\log n)$	B+ tree insert + heap write
Find (PK)	$O(\log n)$	B+ tree search
Update	$O(\log n)$	Heap rewrite + index update
Delete	$O(\log n)$	Tombstone + free list
Range Query	$O(\log n + k)$	k = result size
Secondary Lookup	$O(\log n + k)$	Composite key scan
Cascade Delete	$O(m \cdot \log n)$	m = affected children

Startup Cost

Index rebuild: $O(n \log n)$ where n = total rows

Validation Methodology

Behavioral Validation (Not Just Pass/Fail)

- **Atomicity**: Rollback leaves no partial state
- **Consistency**: FK violations rejected, cascades complete
- **Isolation**: Numeric invariants preserved under contention
- **Durability**: Committed data survives restart

Evidence-Based Approach

Each test documents initial state, operation, and observed final state.

Key Achievements

From In-Memory B+ Tree to Transactional Engine

- **Persistent storage**: Fixed-width heap file with slot recycling
- **Crash safety**: Write-ahead logging with idempotent replay
- **ACID guarantees**: Atomicity, consistency, isolation, durability
- **Referential integrity**: Foreign keys with cascade actions
- **Query processing**: Condition trees, secondary indexes, joins
- **Validation**: 10+ usage scenarios + 5 explicit test cases

Core Principle

Correctness through explicit data layout, operation sequencing, and failure-aware state transitions.